# Ускорение инференса UNet модели

Никитюк Илья, Потапенко Антон, Тюменцева Ирина

31 марта 2026

## 1 Описание проекта

Описание проекта можно найти на странице описания проектов курса ШАД Эффективные модели ML и архитектуры нейросетей.

Цель проекта - ускорение инференса UNet-модели. В качестве бейзлайна рассматривается инференс на PyTorch с вычислениями в формате fp16. Для ускорения инференса относительно бейзлайна применяются: PyTorch torch.compile (JIT-компиляция графа вычислений); компилятор TVM (аппаратно-независимая оптимизация через Relax и TIR), квантизация (снижение точности весов и активаций), спарсификация: pruning / 2:4 semi-structured (разреживание весовой матрицы). Вклад квантизации и прунинга оценивается отдельно посредством профилирования и ablation-study.

В качестве итогов проекта ожидается:

1. Сравнительная таблица latency и throughput для всех рассмотренных конфигураций.

2. Измерение времени инференса после применения оптимизаций с контролем качества (метрики точности) относительно fp16-референса.

3. Публичный git-репозиторий с воспроизводимыми скриптами бенчмаркинга, включая requirements.txt/pyproject.toml и/или Dockerfile (либо ссылка на готовый Docker-образ).

4. Документация в формате Markdown с подробным описанием примененных методов ускорения и полученных результатов.

5. Итоговая презентация, содержащая ключевые выводы по эффективности каждого из подходов.

## 2 Планирование экспериментов

План экспериментов:

1. Построить baseline модели `Unet`
2. Провести эксперименты с использованием компиляторных оптимизаций
   - `torch.compile`
   - `torch.compile` с параметром `max_autotune`
   - `TVM compiler`
3. Провести эксперименты с использованием пост-тренировочных оптимизаций
   - Квантизация (int8, per-tensor)
   - Квантизация (int8, per-channel)
   - Спарсификация (2:4 semi-structured)
   - Спарсификация + квантизация
4. Провести эксперименты с использованием комбинированных методов оптимизаций
   - `torch.compile` + квантизация
   - `TVM` + спарсификация

Текущее состояние проекта в github. Текущий статус: эксперименты 1 и 2 пунктов выполнены.

# 3 Обучение baseline модели

Модель Unet получена дообучением предобученной модели из segmentation models pytorch на COCO-датасете. Обучение проходило с точностью bfloat16. Веса энкодера в процессе обучения не менялись (были заморожены). Оценка качества полученной модели проводилась на валидационном COCO-датасете.

Воспроизвести обучение можно с помощью train_COCO.ipynb

Параметры модели UNet:

- энкодер: ResNet101
- датасет предобучения энкодера: ImageNet
- обучаемые веса: декодер
- всего параметров: 51.524.833, обучаемых из них: 9.024.673

Датасет:

- train: изображения, аннотации
- validation: изображения, аннотации

Параметры обучения:

- Loss: CrossEntropyLoss
- Optimizer: AdamW
- learning rate: 0.001 + scheduler
- Количество эпох: 20

Программное обеспечение:

- python: 3.13.7
- cuda: 13.2
- torch: 2.11

Аппаратное обеспечение:

- CPU: Ryzen 7700x
- GPU: Nvidia 4070 ti Super

Процесс обучения и результаты обучения представлены на изображениях далее.

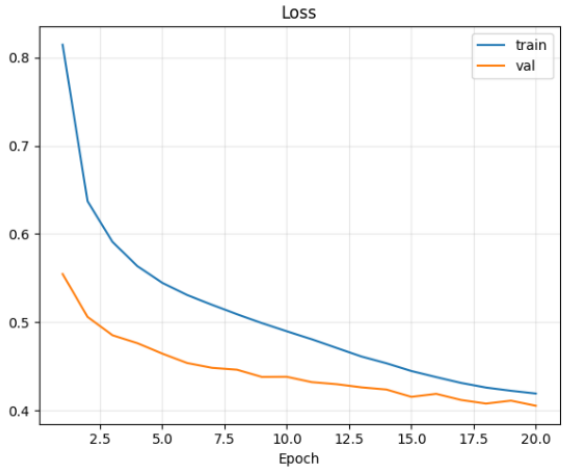


Рис. 1: График изменения функции потерь

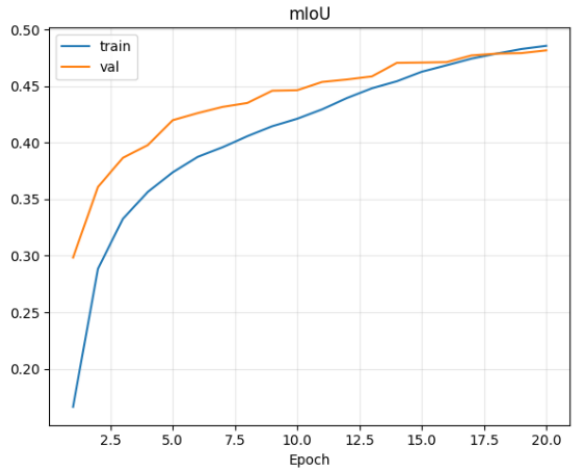


Рис. 2: График изменения метрики качества mIoU

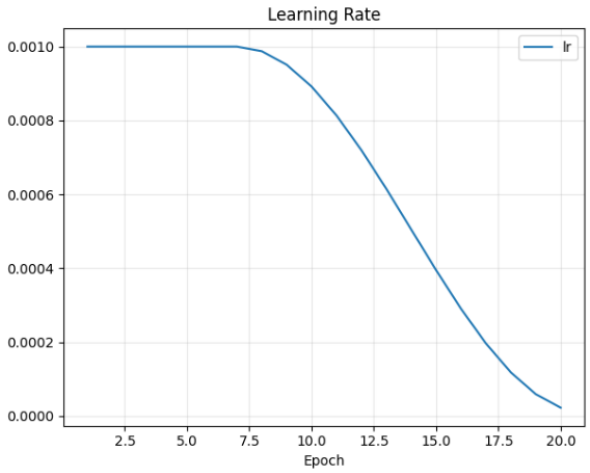


Рис. 3: График изменения шага обучения

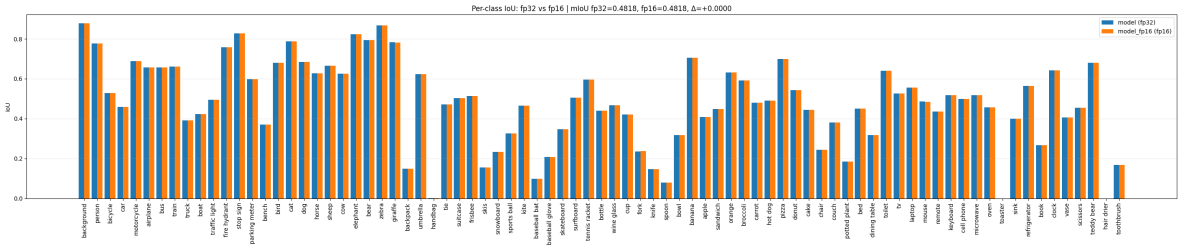


Рис. 4: Значение метрики качества mIoU по классам для модели в fp32 и baseline модели в fp16

Рис. 5: Примеры предсказаний baseline модели

# 4 Эксперименты с compile оптимизациями

## 4.1 Цель эксперимента

Целью эксперимента являлось сравнение нескольких способов выполнения инференса модели сегментации `UNet` с энкодером `ResNet101` на GPU:

- исходная реализация в PyTorch;
- `torch.compile` в стандартном режиме;
- `torch.compile` с оптимизацией;
- TVM Relax.

Сравнение проводилось по трем основным критериям (на A100):

- средняя задержка инференса (Latency);
- пропускная способность (Throughput) при различных размерах батча;
- значение основной метрики качества (`mIoU`).

## 4.2 Полученные результаты

### 4.2.1 Задержка инференса и качество

Таблица 1: Сравнение по задержке и качеству

| **Метод** | **Тип данных** | **Latency, ms** | **mIoU, %** |
|---|---|---|---|
| `baseline` | float16 | 19.596 | 45.2450 |
| `torch.compile (default)` | float16 | 5.697 | 45.2451 |
| `torch.compile (tuned)` | float16 | 2.578 | 45.2450 |
| `TVM Relax` | float32 | 26.708 | 45.2452 |

Из таблицы видно, что:

- наименьшую задержку показал вариант `torch.compile (tuned)`;
- вариант `torch.compile (default)` также значительно превосходит baseline;
- TVM Relax в текущей реализации оказался медленнее baseline;
- значения `mIoU` во всех вариантах практически совпадают, то есть ускорение или замедление не сопровождалось заметной деградацией качества.

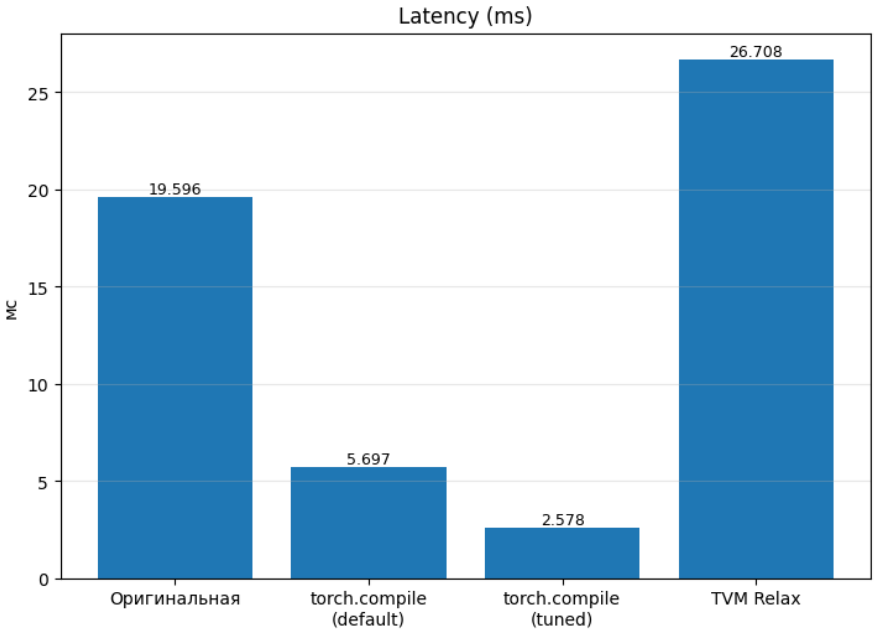


Рис. 6: Latency

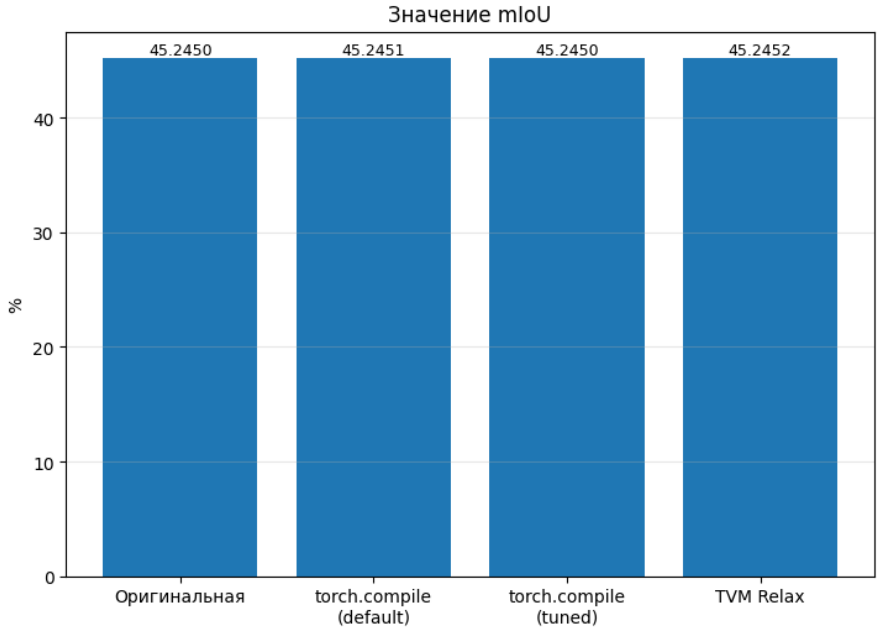


Рис. 7: mIoU

### 4.2.2 Пропускная способность при различных размерах батча

Таблица 2: Пропускная способность, изображений в секунду

| **Метод** | **bs=1** | **bs=2** | **bs=4** | **bs=8** | **bs=16** | **bs=32** | **bs=64** |
|---|---|---|---|---|---|---|---|
| `baseline` | 51.03 | 110.27 | 227.87 | 398.15 | 584.85 | 608.70 | 621.20 |
| `torch.compile (default)` | 175.54 | 299.89 | 547.20 | 874.00 | 972.51 | 1014.05 | 1057.83 |
| `torch.compile (tuned)` | 387.93 | 676.09 | 880.77 | 980.28 | 1075.94 | 1171.47 | 1179.13 |
| TVM Relax | 37.44 | 55.28 | 73.19 | 79.16 | 82.97 | 86.65 | 87.33 |

По пропускной способности наблюдается та же тенденция, что и по задержке:

- `torch.compile (tuned)` является лучшим вариантом практически на всех размерах батча;
- `torch.compile (default)` также существенно превосходит исходную реализацию;
- TVM Relax показывает значительно более низкую пропускную способность, чем все PyTorch-варианты.

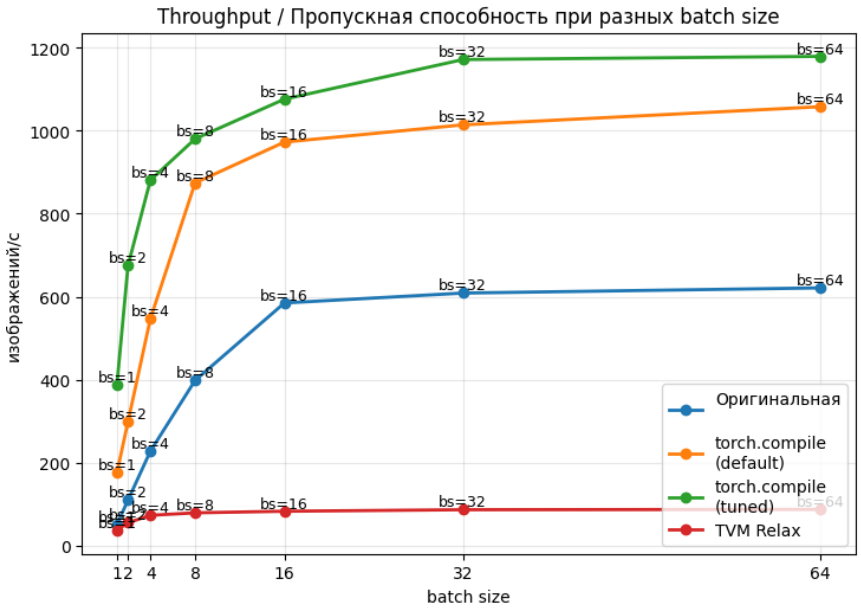


Рис. 8: Пропускная способность

### 4.2.3 Количественный анализ ускорения

Относительно исходной модели ускорение по latency составляет:

$$S_{\text{default}} = \frac{19.596}{5.697} \approx 3.44,$$

$$S_{\text{tuned}} = \frac{19.596}{2.578} \approx 7.60,$$

$$S_{\text{TVM}} = \frac{19.596}{26.708} \approx 0.73.$$

Следовательно:

- `torch.compile (default)` ускоряет инференс примерно в 3.4 раза;
- `torch.compile (tuned)` ускоряет инференс примерно в 7.6 раза;
- TVM Relax в данной конфигурации не ускоряет вычисления, а, наоборот, замедляет их.

Если рассматривать максимальную пропускную способность на `batch_size = 64`, то:

- baseline: 621.20 изображений/с;
- `torch.compile (default)`: 1057.83 изображений/с;
- `torch.compile (tuned)`: 1179.13 изображений/с;
- TVM Relax: 87.33 изображений/с.

## 4.3 Что удалось решить по итогам разработки

В результате проведённой работы удалось:

1. Успешно запустить baseline и `torch.compile`;
2. Собрать TVM с поддержкой CUDA и построить TVM-пайплайн;
3. Выполнить измерение latency и throughput на наборах батчей $1, 2, 4, 8, 16, 32, 64$;
4. Убедиться, что значения `mIoU` остаются практически идентичными во всех вариантах.

## 4.4 Анализ полученных результатов

По результатам экспериментов можно сделать следующие выводы:

1. Для рассматриваемой модели `Unet` и в текущей экспериментальной конфигурации наилучшие результаты по скорости показал `torch.compile`, особенно в tuned-режиме.
2. Вариант `torch.compile (tuned)` обеспечил минимальную задержку инференса и максимальную пропускную способность при всех исследованных размерах батча.
3. TVM Relax в текущей реализации не дал выигрыша по скорости: он уступил как baseline, так и обоим вариантам `torch.compile`.

4. При этом качество сегментации во всех вариантах осталось практически одинаковым, так как значения `mIoU` отличаются лишь на уровне тысячных и десятитысячных долей процента. Поэтому, основное различие между подходами определяется не качеством инференса, а исключительно эффективностью исполнения.

* Стоит отметить, что при реализации TVM-пайплайна возникли проблемы с корректной работой данного компилятора:

- Запуск TVM удалось произвести на `float32`, что в свою очередь увеличивает задержку инференса;
- Также часть возникших конфликтов между CUDA и TVM не до конца удалось решить, поэтому некоторые операции расчетов модели не были оптимизированы. В связи с этим, были получены не самые оптимальные результаты.

# 5 Эксперименты с посттренировочными оптимизациями

**В планах**

# 6 Эксперименты с комбинацией оптимизаций

**В планах**

# 7 Результаты

В таблице представлены группы экспериментов по измерения end-to-end latency и throughput для всех конфигураций проекта. Качество модели будет оцениваться относительно fp16 baseline с использованием метрики IOU.

Таблица 3: Группы планируемых экспериментов

| **Конфигурация** | **Latency (ms)** | **Throughput (img/s)** | **IoU (%)** |
|---|---|---|---|
| *Baseline* | | | |
| PyTorch fp16 (baseline) | 19.596 | 51.03 | 45.2450 |
| *Компиляторные оптимизации* | | | |
| PyTorch torch.compile (mode="default") | 5.697 | 175.54 | 45.2451 |
| PyTorch torch.compile (mode="max-autotune") | 2.578 | 387.93 | 45.2450 |
| TVM (Relax/TIR) | 26.708 | 37.44 | 45.2452 |
| *Пост-тренировочная? оптимизация* | | | |
| Квантизация (int8, per-tensor) | | | |
| Квантизация (int8, per-channel) | | | |
| Спарсификация (2:4 semi-structured) | | | |
| Спарсификация + квантизация | | | |
| *Комбинированные методы* | | | |
| torch.compile + квантизация | | | |
| TVM + спарсификация | | | |